addac112 LOOPS

This is the addac112 looper firmware with a different engine in the left knob section: **LOOPS**, a port of the Looops plugin (`../PLUGINSto112/Loops 3`).

LOOPS is a chance sampler. It keeps two seconds of sound and turns them into loops with three layers. Every flip of **DICE** deals a new loop: each layer's place in the recording, its speed, direction, notes, place in the stereo field and echo send, all generated from a loop number.

Each layer has one of three modes:

- **TAPE** (the plugin's *Loop*) goes round its region like a tape loop. Slower is lower, and the speed can be played live.
- **STRETCH** plays the region slowly as aligned grains that keep the pitch.
- **SPARKLE** picks short grains from the region, in sparse bursts.

Then each layer goes through its own speaker (direct, radios, telephone, megaphone, tin can...) and travels across the stereo field, into a ping-pong echo and a reverb wash.

It was built from NULLSTATION (`addac112_nullstation`), with the RADIO engine replaced by LOOPS. The looper, banks, SD card, presets, loop pitch, sync and REC prob/delay are the same as in the stock firmware.

The plugin parts the 112 leaves out:

- the **Chance** section (Fate, Seed, Auto, Odds): the DICE switch deals a new loop, as Chance does in Random fate;
- **per-layer wash**: every layer goes into the wash;
- the **synced echo**: DELAY TIME is always free.

```
engine/     loops_engine.* (DSP), analysis.* (recording analysis, pitch), speaker.h, dsp.h,
            loops_param_map.h (knobs, settings, preset keys); no firmware dependencies
reference/  the plugin's engine, presets and tests, unchanged (the A/B reference)
test/       desktop tests of the engine: ./run_tests.sh; Cortex-M7 cycle estimate: m7/
firmware/   the 112 firmware (src/, web/ editor, Makefile)
banks/      BANK0_LOOPS: the plugin's 16 factory presets as a 112 bank
tools/      make_factory_bank.sh: builds it from reference/Presets.h
aux/        the labelled panel (panel_loops.png), the panel's functions (panel_funcs.jpeg)
and the plugin's interface (loops3_panel.png)
```

```
cd firmware
make                # build/LOOPS_<version>.bin (git hash, or the build date)
make program-app    # bootloader in DFU mode
```

It builds against addac112's libdaisy / DaisySP / CMSIS (`make ADDAC112_DIR=...` to override).

Switching engines

Each engine is its own firmware. This one always runs LOOPS. Plain granular is the stock addac112 firmware.

1. Put the firmware `.bin` files (112, CD SKIP, FRIPPER, NULLSTATION, LOOPS, ...) in a folder named `ENGINES` on the SD card.
2. Go to **Menu** → **ENGINE** (the last item of the main menu). It lists the `.bin` files in `ENGINES`.
3. Choose one and confirm **FLASH & RESTART? YES**.

Presets store both engines' values: the stock grain keys plus `loops_*` keys. The stock 112 firmware ignores the `loops_*` keys, but if it saves a preset, those keys are dropped from that preset.

LOOPS controls (dual-label overlay)



The engine's labels are in orange; the rest of the panel works as on the stock 112.

[L]: the knob edits the layer the LAYER switch points at.

Row	Col 1	Switch	Col 2	Col 3
1	POSITION / **LENGTH** [L]	START GRID / **LAYER** (1 / 2 / 3)	POS DEV / **WANDER** [L]	GRAINS ACTIVE / **LAYER 1 VOLUME**
2	PLAY LENGTH / **POS** [L]	SIZE GRID / **MODE** (TAPE / STRETCH / SPARKLE)	LENGTH DEV / **REVERSE** [L]	ATTACK / **LAYER 2 VOLUME**
3	GRAIN DELAY / **SPEED** [L]	SKIP CLOCK / **PARAM** (1 / 2 / 3)	DELAY DEV / **PARAM** [L]	DECAY / **LAYER 3 VOLUME**
4	DIRECTION / **DELAY TIME**	REPEAT / **DICE** (OFF / ON)	PROB/REPEAT / **FEEDBACK**	GRAIN PITCH / **DELAY MIX**
5	VOLUME MIN / **SPREAD**	–	VOLUME DEV / **TRAVEL SPEED**	PANNING / **WASH MIX**

PARAM edits one of the layer mode's own parameters (the right side of each layer in the plugin), picked by the PARAM switch:

	PARAM 1	PARAM 2	PARAM 3
TAPE	GLIDE: how fast the speed slides to a new value or note, instant to 10 s	FADE: the crossfade where the tape goes round, 2 ms to 1 s	FREE
STRETCH	GRAIN: 20 ms to 1 s	SMOOTH: grainy to seamless	FREE
SPARKLE	RATE: 0.5 to 40 grains/s	SIZE: 5 to 400 ms	BURST: the chance of quick clusters

Knobs

- **POS**: where the layer's region starts in the two seconds. DICE deals a new one to every layer.

- **LENGTH**: the layer's region, 5 ms to 2 s (300 ms at the centre). Each loop deals a variation around it. Under about 20 ms a TAPE layer becomes a pitched hum.
- **WANDER**: the plugin's POS DEV and LEN DEV together. The region's start and length wander over time, up to a second and two octaves either way. WANDER TIME (layer settings) sets how often they head somewhere new.
- **REVERSE**: the chance a tape pass or grain plays backwards. A detent of ±2 % around the centre is exactly 50 %: **ping-pong**, forward and backward in turn, with no chance (`PING–PONG` on the display).
- **SPEED**: the speed of the layer's mode:
  - TAPE: 1/16x to 2x, live (the pitch), gliding by GLIDE;
  - STRETCH: 1/16x to 1x;
  - SPARKLE: each grain's speed, so its pitch, 1/16x to 2x.

  It snaps to whole divisions (1, 1/2, 1/3 ... 1/16, or 2x) unless FREE is on for that mode. DICE deals a new one to every layer.
- **FREE** (PARAM 3 on TAPE and STRETCH): two steps. In the lower half of the knob SPEED snaps (`SNAP`), in the upper half it moves freely (`FREE`). Its pickup takes any position on the side of halfway the value is on. SPARKLE's FREE is in the layer settings.
- **LAYER 1 / 2 / 3 VOLUME**: each layer's level.
- **DELAY TIME** (20 ms to 1.5 s, 300 ms at the centre), **FEEDBACK** and **DELAY MIX**: the ping-pong tape echo. Each layer's send is dealt with the loop. DELAY MIX is the GRAIN PITCH knob plus its CV jack.
- **SPREAD**: how wide the layers travel.
- **TRAVEL SPEED**: how fast each layer glides to a new place in the stereo field: an 8 s drift fully left, a 20 ms sweep fully right.
- **WASH MIX**: the reverb haze.
- LENGTH and SPEED move in steps larger than the pot noise (0.3 % of the travel), so a TAPE layer's pitch does not wobble.

Switches

- **LAYER**: which layer the [L] knobs edit. When it changes, those knobs wait until they cross the new layer's values (`PICKUP` on the display).
- **MODE**: sets the focused layer's mode when the switch **moves**. Changing LAYER does not change any layer's mode, so the switch can point at another mode than the focused layer's (the display shows the layer's). The layer's other values stay, so going back restores the old mode as it was. SPEED and PARAM then wait for pickup.
- **PARAM**: which parameter the PARAM knob edits (it then waits for pickup).
- **DICE**: going from OFF to ON deals a new loop. Each layer also gets a new POS and a new SPEED (for each mode), from the loop number. The knobs then wait to pick them up. Staying ON does nothing more; the display's `P1` field is inverted while it is ON.

Other panel controls in LOOPS mode

- **GRAINS VOLUME** is the LOOPS level.
- **GRAINS FEEDBACK** records LOOPS into the loop.
- **GRAINS trigger out** sends a pulse on each new loop (TRIGGER OUT +EVENTS: also on every tape pass and grain).
- **Encoder + STOP** (special function, enabled in Options) is **RECORD**: it captures two seconds (CAPTURE: MANUAL), or arms / disarms ON LEVEL and NEXT BAR.

Time While a loop plays, its length is one bar of BEATS / LOOP beats (default 4), and its start is the downbeat, whatever the loop pitch or direction. This drives NEXT BAR. With no loop playing, the beat is the TEMPO setting.

Settings

- Two quick pages, scrolled from the main screen, last after the OTHER SETTINGS pages:
  - **LOOPS SETTINGS**: CAPTURE, SOURCE, FEEDBACK, SCALE, RANGE, DIRECTION, VOICES, CHANGE.

    - **LOOPS LAYER n**: the settings of the layer the LAYER switch points at: SPEAKER, WANDER TIME, SPARKLE FREE, MUTE.
- Every setting, by group, under **Menu** → **Options** → **10.LOOPS OPTIONS** (the last menu pages):
    - **CAPTURE**: SOURCE (LIVE / LOOP as heard / BOTH), CAPTURE (MANUAL / ON LEVEL / NEXT BAR / ALWAYS), THRESHOLD, INPUT, MIC, FEEDBACK (overdub), NORMALISE.
    - **PITCH**: the notes the layers play, counted from the recording's pitch:
        - SCALE: octaves, fifths, major / minor triad, pentatonic, major, minor, chromatic, or FREE (any pitch, between the notes too);
        - RANGE: how far from the recording's pitch, 0 (off) to 24 semitones;
        - DIRECTION: up, down, or both ways;
        - VOICES: notes per layer, 1 to 4. STRETCH and SPARKLE play them as chords; a TAPE layer plays one per pass;
        - CHANGE: how often a note moves to a new one while the loop plays (0 %: only on DICE);
        - ROOT: retunes the recording to a key by its detected pitch.
    - **SPACE**: TRAVEL DEV (each glide's time varies, up to 4x either way), CLICKS, CLICKS POST.
    - **TIME**: BEATS / LOOP, TEMPO, TRIGGER OUT.
    - **LAYER 1 / 2 / 3**: SPEAKER (13 models), WANDER TIME (0.25 to 16 s), SPARKLE FREE, MUTE.

Display (LOOPS)

- **Main screen:**
    - A header with the focused layer and its mode, and the loop number. It shows `REC nn%` while recording and `ARMED` while waiting.
    - A lane per layer:
        - its region of the two seconds, thicker for the focused layer, dotted for the others;
        - its grains (and tape playhead) as dots above it;
        - a dot at the left edge on each event;
        - a dotted line when the layer is muted.
    - The waveform of the recording. While recording, the new one is drawn up to where it is, with the old one dotted after it.
- **Left bar:** the input level (–60 to 0 dB), with a line at THRESHOLD in ON LEVEL. While recording: the progress, blinking.
- **Right bar:** SPREAD.
- **Top bar:** the focused layer's mode (`TP` / `ST` / `SP`).
- **Bottom bar:**
    - the PARAM switch (`P1` / `P2` / `P3`), inverted while DICE is ON;
    - a meter per layer (the focused one filled);
    - N = the note the focused layer plays, in semitones from the recording (then L, the loop pitch, as in the stock firmware).
- **Readout:** turning a knob or flipping a switch shows its name and value for 1.5 s, with `PICKUP` until the knob reaches the value.

The recording

- **SOURCE**: what a capture records: the LIVE input, the LOOP as you hear it, or BOTH. VOL DRY stays clean.
- The recording is two seconds, mono.
    - It is normalised (NORMALISE, up to +30 dB).
    - Its DC is removed and its ends fade over 4 ms.
    - Its onsets are found: SPARKLE grains start on them.
- **FEEDBACK** is overdub: the recording that was playing is added in at this level, scaled back if the sum would clip.
- **CAPTURE ALWAYS** keeps recording, replacing the sample every two seconds while the layers play on.

- The analysis of a finished recording is spread over the next blocks: over the capture under way with CAPTURE ALWAYS (~1.75 s), otherwise within about a second. The new recording comes in after that, crossfaded; TAPE passes carry on from the same place in it, one layer per block (each looks for the best place to join, so three at once would be a spike).
- No layer copies the recording: the three recordings sit in SDRAM, and each grain or tape pass is only which one it reads and where its playhead is.
- **ROOT** retunes the recording to a key by its detected pitch. The pitch is only tracked when ROOT is set.
  - A new recording's pitch is tracked before it comes in, so it plays in the key from its first sample: with CAPTURE ALWAYS over the capture under way (the analysis has the first half of it), otherwise within about a second.
  - A recording that played untracked (ROOT set later) is tracked while it plays, within about a second.

Sound against the plugin

The engine is the plugin's DSP, with:

- caller-owned memory (SDRAM) instead of the heap;
- no locks;
- fast sine, pow and log on the audio path;
- the 112 additions, each of which can be switched off (`Params::exactAU`);
- POS (`LayerParams::pos`): below 0 the region starts where the loop deals it, as in the plugin.

The tape motor's glide (log speed, every sample in the plugin) keeps the plugin's `log` / `exp` even with the fast math: the playhead integrates the speed, so an approximation drifts the phase of every pass. The 112 only computes it while the motor moves.

`test/run_tests.sh`:

- **ab_test** renders 43 cases on both engines with the same input, seed and loop number: each type on each layer, the 16 factory presets, every capture mode, overdub, DICE, pitch (scales, directions, voices, change, root), ping-pong, glide, echo, travel dev, clicks, mic. What the 112 leaves out is set to the plugin's defaults on both (`test/loops_host.h`).
  - With the plugin's arithmetic (`–DLOOPS_FAST_MATH=0`): identical, sample for sample.
  - With the fast math the 112 runs: –74 to –141 dB below the signal, the level the same to 0.1 dB.
    - The exception is *Hum the root*: the detected pitch moves by rounding, so the retuned phase drifts. The level and zero crossings are the same.
  - With the 112 additions on: the same level (±0.8 dB) and brightness.
- **The plugin's own tests** (`reference/engine_test.cpp`, through `test/plugin_api/`) pass on the 112 engine, both as the plugin and with the 112 additions on. `test/plugin_test_112.patch` takes out the checks of what the 112 does not have (Fixed fate, per-layer wash) and presses DICE for Chance. They cover:
  - pitch detection, types, mute and deviation;
  - Chance and capture modes;
  - save / restore, pitch (scales, range, voices, change), tape speed and glide, travel and wash;
  - ping-pong, travel dev, overdub, Sparkle speed, clicks and dips;
  - all 13 speakers.
- **finisher_ref_test**: the 112's analysis of a recording (two passes, its mean and peak from the recording itself) against the plugin's `finishRecording` (eight passes), bit for bit, at three rates, four kinds of material, with and without normalise and overdub, however it is sliced.
- **features_test** covers the 112 additions:
  - the sliced analysis gives the same buffer and onsets;
  - TAPE layers hand over to a new recording one per block, all within a few blocks, with no larger steps than the plugin's;
  - a paced analysis spreads over the capture under way and is always done before it ends;
  - the budgeted pitch tracker gives the same note as the plugin's, to 0.01 semitone (each frame stops at its pitch's lag; each lag is summed in four parts);

    - paced pitch tracking never does more than 128 multiply-adds per sample in a block, even when ROOT is set late in a capture;
    - the 112's faster alignment search picks the plugin's offsets (2000 random cases);
    - the worst block when a recording ends is several times shorter than the plugin's on desktop;
    - pitch is tracked only with ROOT;
    - the grain cap holds without clicks;
    - POS sets the region's start (and below 0 leaves it to the loop);
    - DICE deals one loop per rising edge;
    - the tape motor rests exactly on its target;
    - half rate at 96 kHz;
    - buffer load.
- **rate_test** covers every sample rate of the 112 (8 to 96 kHz).
- **param_map_test** checks the knobs, settings and preset keys.
- The web editor tests check that the editor's `loops_*` keys and defaults are exactly the firmware's.

A bug in the plugin, fixed in the port and applied to the reference for the A/B (`test/plugin_wrap_fix.patch`):

- Where: the echo, the travel delay line and the speaker comb.
- What: `rp = w - t` can round to exactly the line's length once wrapped, and the plugin then reads one sample past the end.
- What you hear:
    - in the echo, a sample of leftover heap memory (the plugin's output changes from run to run);
    - in the travel line, a one-sample dropout.
- The fix is three lines; worth applying to the plugin too.

CPU

The engine files are built with `-O2`, and so are the looper's (`granular.cpp`: the audio callback and mixdown; `loop.cpp`: its sample reads and writes), which share every audio block with the engine. The rest of the firmware, like the stock 112, is built with `-Og`.

On the 112 itself, measured with the CPU readout (48 kHz, a new bank, the default config, a ~1 s loop recording and CAPTURE ALWAYS: the heaviest block of each half second):

Case	CPU	Engine
defaults (three Loop layers)	74–85 %	51–55 %
VOICES 4, CHANGE up	75–85 %	53–57 %
three Stretch layers	73 %	50 %
three Stretch layers, ROOT set	80 %	54 %
three Stretch layers, ROOT set, MEGAPHONE	85 %	60 %

The 112's looper, mixdown and the rest of the callback take the other ~25 %. The Cortex-M7 runs the engine about twice as slowly as an instruction count says (FPU latency, SDRAM): the emulator's figures below are for comparing cases.

What took it there:

- **ITCM**: the engine's and the looper's per-sample code (`LOOPS_HOT` / `ITCM_HOT`) and all of libm (~8 KB) run from ITCM, zero-wait, outside the 16 KB instruction cache (`firmware/STM32H750IB_sram_itcm.lds`; `main.cpp` copies it at boot, from `.preinit_array`). ITCM starts at 0x400, so a stray null-pointer write misses the code.
- **libdaisy is built at `-O0`** (`DEBUG=1`) in the vendored copy the stock 112 and the other firmwares link. LOOPS links its own `-O2` build instead (`firmware/build_libdaisy.sh` into `firmware/libdaisy_O2/`, made by `make` when missing; the vendored one stays as it is). The looper's per-sample code (mixdown, rec delay, trigger outs) also reads the time, the sample rate and the hardware version once per block, not several times a sample.

- **`hw.GetUs()` wraps every ~17.9 s** (a 32-bit tick over 240), so a block timed across the wrap read ~71 minutes: an overrun, `!!` and a grain cut every ~18 s, with nothing wrong. The audio callback is timed in CPU cycles.
- **Grains play a block at a time** (`renderGrain`): each grain runs through its part of the block with its state in registers, and the grains are brought up to date (`syncGrains`) only where the layers read or change them (a spawn, a type change, a recording starting, a tape pass near its end; a Loop layer follows its pass's distance itself in between). The same grains, sample for sample (the exact A/B is still identical), at about half the cost on the M7: playing them all every sample left it waiting on every grain's chain of double and float operations.
- **Speakers process the block a stage at a time** (each biquad, the comb, drive, rattle, hold, crush), their state in registers; the converter's crush only when its held sample changes. The same output.
- **The tape motor's glide arrives**: in float, exp(log) stalls short of the target (up to 0.6 % off, ~10 cents, with the longest GLIDE) and the plugin then goes on computing it every sample. The 112 snaps to the target once it stops moving.
- **Pitch tracking** (ROOT) is paced like the analysis (at most 128 multiply-adds per sample, ~10 % of the M7), each frame stops once its pitch's dip is found (a 220 Hz note: 56 of 218 lags), and each lag is summed in four parts.
- **Stretch grains' alignment** reads its window as indices worked out once per spawn and scores two candidates at a time: the same offsets.

The emulator (`test/m7/m7_cycles.py`, 128-sample blocks): executed instructions, with VDIV/VSQRT weighted at 14 cycles. It ignores SDRAM, cache and FPU stalls.

Case (48 kHz, the grain ceiling: 20)	Average	Worst block
no recording yet	~21 %	~22 %
defaults (three Loop layers)	~29 %	~36 %
+ heavy speakers	~33 %	~40 %
+ VOICES 4, CHANGE	~37 %	~42 %
three Stretch layers	~40 %	~45 %
ROOT set, CAPTURE ALWAYS	~42 %	~47 %
Dense	~34 %	~45 %
Slow pad (Tape / Stretch / Sparkle)	~39 %	~43 %
worst: 2 Sparkle storms + Stretch (4 voices), 3 heavy speakers, mic, clicks post (47 grains, uncapped)	~76 %	~83 %

A block that runs over its time is a click on **every** output (the dry input too), whatever the volumes.

- **Grain ceiling**: 20 grains at once (12 above 60 kHz). Uncapped, the worst case reaches 47 grains.
- **CPU guard**, from each block's load (its time over the block period), not an average:
  - above 75 %, the cap drops at once to three quarters of the grains playing; above 90 %, to half of them (down to 8);
  - it goes back up 2 at a time, after 250 ms with every block below 60 %;
  - new grains are skipped; playing ones are never cut, and a tape pass is never refused (the pass before it is already fading), so the guard itself never clicks.
- **Overruns on the display**: `!!` (under the top bar, right) stays for a second after any block above 90 % of its time. The readout box shows each overrun (a block over its time, or the callback late): `OVERRUN <count> <total>% E<engine>` (or `OVR <count> L<late>`) and the engine by part, `L G T A P`, of the worst block since it was last shown.
- **The analysis of a recording is paced to its deadline**: spread evenly, with a 25 % margin, over the capture now under way (~1.75 s with CAPTURE ALWAYS), or else over a second (a single capture comes in within ~1 s). On the 112 its pass through SDRAM costs several times what the emulator shows, and evicts the cache the rest of the engine uses: at the old pace (~0.2 s) it added ~25 % to its blocks. With ROOT set the analysis has half the time and pitch tracking the rest. (The desktop tests and the emulator bench run it at the old pace: `Params::analysisBudget` 0.)

- Without overdub (FEEDBACK at 0) the analysis is a single pass over the recording: DC removal, normalising, fades, overview and onset energy together (with overdub, two: the ceiling needs the sum first). The same result, bit for bit. A low E % with an overrun means the time went outside the engine (the looper, the main loop holding audio off).
- **Tape turnarounds**: a pass that reverses (REVERSE above 0 %, ping-pong) waits up to 20 ms for a peak in the waveform. The plugin scans ±6 ms of the recording for it on every sample it waits, about 600 reads per sample: a spike at the end of every reversing pass, heard as a click. The port keeps the window's peak from sample to sample and only reads what enters it: the same result, sample for sample.
- **SAMPLERATE goes up to 48 kHz** in LOOPS: at 96 kHz the 112's looper (it runs every sample, at the full rate) and the engine together don't fit in a block, and the module froze. The menu and the web editor offer 8–48 kHz; a preset or setting saved at 96 kHz plays at 48. (The engine itself still runs at half rate above 60 kHz, with 47-tap half-band filters in and out from NULLSTATION; the desktop tests keep covering it.)
- **The CPU readout** (off in releases: `kCpuAlways` in `gui.cpp`) stays in the box on the main screen whenever no knob or switch readout is shown: the heaviest block of each half second, in % of the block. With it, the box shows the CPU clock, the caches and whether SDRAM is cached for a few seconds after boot.
  - `CPU <total>% E<engine> [REC]`: REC while a recording is analysed.
  - `L<layers: spawns, tape> G<grains> T<travel: speakers, Haas> X<echo, wash, output> A<analysis and pitch tracking>`: the engine by part (`Engine::profile`, `LOOPS_PROFILE` in the Makefile); the rest of E is input and recording.
- Port optimisations, each without changing the plugin-exact output (or only under the fast math):
  - the Wander curve once per block, not three `pow` per sample;
  - the Stretch speed once per block;
  - the tape motor's glide only on Loop layers, and only while it moves;
  - the tape turnaround's peak search kept from sample to sample (see above);
  - the display snapshot walks only the used part of the grain pool;
  - reciprocal glides and envelopes;
  - the region walkers stepped every 16 samples (112 mode only);
  - the speaker drive without `sqrt` and division, and the telephone's mu-law with fast log / exp;
  - a recording's analysis in two passes over SDRAM, not eight (its mean, lowest and highest sample are kept while it records), and its onset picking sliced over blocks;
  - the search for where a grain or pass joins (align) reads the recording without wrapping the index when the window is clear of its ends; Stretch grains search against the recording the grain before plays (112 mode only: after a swap the plugin compares against the new one).

Memory

	GRANULAR (stock)	LOOPS
Loop memory	360 s	~364 s (mono at 44.1 kHz)
Engine buffers (2.6 MB: three 2 s recordings, echo, wash, analysis, display copy)	–	top of the loop array in SDRAM
Grains, layers, speakers, block buffers + bridge (~90 KB)	–	heap (RAM_D2), created at boot
Grain voice reserve (21 KB heap)	yes	no

- DTCM is at 89.4 % (stock: 88.6 %); ITCM (the hot code and libm) at 92 % (59 of 63 KB).
- The binary is 387 KB of the 480 KB the bootloader copies to SRAM.

Factory bank

`banks/BANK0_LOOPS` holds the plugin's 16 factory presets as 112 presets (`1.CFG` to `16.CFG`).

- To use it, copy the folder to the SD card. The 112 reads it as bank 0 (only the number after `BANK` counts); rename it if bank 0 is taken.
- It has no loops: record a sample (Encoder + STOP), or add WAV files to its `WAV` folder.
- `NAMES.txt` lists the presets by number, and what a plugin preset does that the 112 leaves out:
  - *Endless replace*, *Fixed fate, bar by bar* and *Dice speeds* deal new loops with Auto (and Fixed fate): flip DICE instead;
  - *Oval skips* sends less of layer 1 into the wash;
  - *Glass sparkle* syncs the echo: DELAY TIME holds its dotted eighth at 120 BPM;
  - *Scattered voices* and *Dice speeds* turn dice on for some layers: DICE deals speeds to every layer.
- POS is left to the loop (–1): each layer starts where the loop deals it, and the knob picks it up from there.
- Each preset sets every `loops_*` value. The rest (volumes, loop pitch, options) comes from the bank's defaults.
- `tools/make_factory_bank.sh` rebuilds it from `reference/Presets.h` with the firmware's own preset writer, and checks that every file reads back the same.

Web editor

`firmware/web/` has four Loops sections next to Grains: Loops (the global values) and Loops layer 1–3. Each value has its help text.

Run the editor's tests with `for t in firmware/web/test/*.test.js; do node "$t"; done`. The sample files are read from `../addac112/support`. The tests check two things:

- the editor's `loops_*` keys are exactly the ones `loops::saveKeys()` writes;
- its defaults are the engine's (`test/run_tests.sh`).

Not done yet

- Not yet run on hardware. Things to check:
  - CPU % in the worst cases above, and the guard;
  - heap headroom with large banks (the bridge is ~22 KB more than NULLSTATION's);
  - the pot deadband on LENGTH and SPEED, and the REVERSE detent;
  - LAYER, MODE and PARAM pickup feel;
  - which way the REPEAT switch reads as DICE ON (the bridge takes the switch's 1 as ON);
  - Encoder + STOP RECORD, the trigger out;
  - NEXT BAR timing against the loop;
  - the display at 50 fps.